

SOLID Design Principles in C# with Real-Time Examples

In this SOLID Design Principles in C# with Examples article series, I will discuss the examples using real-time scenarios using different types of .NET Applications, including MVC, Web API, and Console Applications.

The SOLID Design Principles in C# help us solve most software design problems. These principles provide multiple ways to remove the tightly coupled code between the software components (between classes), making the software designs more understandable, flexible, and maintainable.

This SOLID Design Principles Course is For whom:

This course is designed for Students, Beginners, Intermediates, and Professional Software Developers who want to learn SOLID design principles step by step with real-time examples. This tutorial provides a hands-on approach to the subject with step-by-step programming examples that will assist you in learning and putting the acquired knowledge into practice.

Why Do We Need to Learn SOLID Design Principles?

As developers, we start developing applications using our experience and knowledge. But over time, the applications might cause bugs. We must alter the application design for every change request or new feature request. After some time, we might need to put in a lot of effort, even for simple tasks, which might require the full working knowledge of the entire system. However, we can't blame the change requests or new feature requests as they are part of software development. We can't stop them, and we can't refuse them either. So who is the culprit here? Obviously, it is the Design of the Application.

So, by considering the above problem and helping Students, Beginners, and Professionals Software Developers who want to learn How to design good software using SOLID Design Principles using C# Language in a quick time, I have decided to start a series of articles on SOLID Design Principles with Real-time Examples.

What are the Main Reasons Behind Most Unsuccessful Applications?

The following are the Main Reasons Behind Most Unsuccessful Applications.

1. Putting More Functionalities on Classes. (In simple words, we put many functionalities into the class even though they are unrelated to that class.)
2. Implementing Tight Coupling Between the Software Components (i.e., Between the Classes). If the classes depend on each other, changing one class will also affect the other.

How to Overcome the Unsuccessful Application Development Problems?

1. We need to use the Correct Architecture (i.e., MVC, Layered, 3-tier, MVP, and so on) as per the Project Requirements.
2. As developers, we must follow the Design Principles (e.g., the SOLID Principles, the ONIO Design Principles, etc.).
3. Again, we must choose the correct Design Patterns (Creational Design Pattern, Structural Design Pattern, Behavioral Design Pattern, Dependency Injection Design Pattern, Repository Design Pattern, etc.) per the project requirements.

What are SOLID Design Principles?

The **SOLID Design Principles** are used to manage most of the Software Design Problems we, as developers, generally encounter in our day-to-day programming. These design principles are tested and proven mechanisms to make software designs more understandable, flexible, and maintainable. As a result, if we follow these principles while designing our applications, we can develop better applications.

SOLID Design Principles represent five Design Principles used to make software designs more understandable, flexible, and maintainable. The Five SOLID Design Principles are as follows:

1. **S** stands for the **Single Responsibility Principle**, also known as **SRP**. This Principle states that each software module or class should have only one reason to change. In other words, each module or class should have only one responsibility.
2. **O** stands for the **Open-Closed Principle**, also known as **OSP**. The Open-Closed Principle states that software entities, such as modules, classes, functions, etc., should be open for extension but closed for modification.
3. **L** stands for the **Liskov Substitution Principle**, also known as **LSP**. This Principle states that the object of a derived class should be able to replace an object of the base class without causing any errors in the system or modifying the behavior of the base class. That means the child class objects should be able to replace parent class objects without changing the correctness or behavior of the program.
4. **I** stand for the **Interface Segregation Principle**, also known as **ISP**. This Principle states that Clients should not be forced to implement any methods they don't use. Rather than one fat interface, numerous little interfaces are preferred based on groups of methods, with each interface serving one submodule.
5. **D** stands for **Dependency Inversion Principle**, also known as **DIP**. This Principle states that high-level modules/classes should not depend on low-level modules/classes. Both should depend upon abstractions. Secondly, abstractions should not depend upon details. Details should depend upon abstractions.

Applying the SOLID principles in our Application Development can make our application code more modular, easier to understand, and less error-prone when changes are made. Applying these principles enables us to create software that is simple to maintain and expand and can adapt better to future changes.

In this article series, I will cover all the features of **SOLID Design Principles in C#** with real-time examples. As we progress, you will learn from the basic to advanced level features of SOLID Design Principles in C# with Real-time Examples.